

LLM-Based Knowledge Graph Construction and Retrieval for Legal Document Analysis

Samuel Bagín, Marek Vančo

Faculty of Electrical Engineering and Information Technology

Slovak University of Technology

Bratislava, Slovakia

xbagins@stuba.sk, marek_vanco@stuba.sk

Abstract—This paper presents a system for automated extraction and linking of information from legal texts into a knowledge graph with advanced semantic search. The system leverages large language models (LLMs) to transform unstructured legal documents into structured data stored in a Neo4j graph database, combined with a vector store for hybrid retrieval. A hybrid extraction approach combining open-domain detection with schema-guided extraction achieves consistent and comprehensive knowledge graphs. The GraphRAG-based search pipeline enables users to pose questions in natural language and obtain precise answers grounded in structured relationships and semantic similarity. We evaluate the processing pipeline and search accuracy on test documents, demonstrating the effectiveness of the hybrid approach.

Index Terms—knowledge graph, large language models, legal document analysis, GraphRAG, Neo4j, information extraction

I. INTRODUCTION

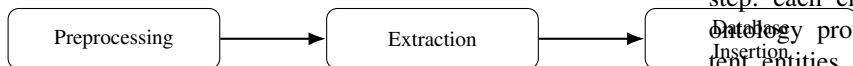
With the growing use of Large Language Models (LLMs) across various domains, there is an increasing need for effective methods to analyze and process specific document types such as legal texts. LLMs alone often struggle with the complex structures and relationships in legal documents, leading to hallucinations and limitations in providing precise information [1].

We propose using LLMs to transform unstructured text into structured data stored in a knowledge graph (KG), subsequently used to answer questions via GraphRAG (Graph Retrieval-Augmented Generation) [2]. The system combines a Neo4j [3] graph database with a vector store for hybrid search, enabling users to ask questions in natural language and receive precise answers based on both structured relationships and semantic similarity.

The remainder of this paper is organized as follows: Section II describes the document processing pipeline, Section III presents the search and retrieval mechanism, Section IV covers testing and results, and Section V concludes the paper.

II. DOCUMENT PROCESSING

The processing pipeline transforms a PDF legal document into a structured knowledge graph through three main stages: preprocessing, extraction, and database insertion.



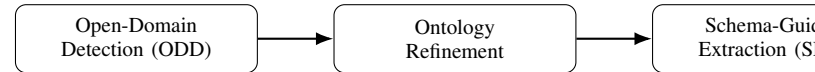
A. Preprocessing

The PDF document is loaded, classified, and split into chunks. Classification is performed by Google Gemini-2.5-Flash, which determines the document type (contract, law, decree, etc.) and legal domain (constitutional, financial, civil, etc.) from the first two pages. Each classification includes a confidence percentage (0–100%).

The text is then split into chunks using `RecursiveCharacterTextSplitter` from LangChain [4]. For open-domain extraction, chunks are 1200 tokens with 200-token overlap; for schema-guided extraction, 1024 tokens with 128-token overlap.

B. Extraction

We employ a hybrid extraction approach based on the ODKE+ study [5]. Pure open-domain extraction yields rich but inconsistent graphs with duplicates and fragmentation (e.g., `Place` vs. `Location` as separate types). Pure schema-guided extraction risks missing entities not defined in the ontology. The hybrid approach combines both, achieving 98.8% extraction accuracy [5].



1) *Open-Domain Detection (ODD)*: Each chunk is processed in parallel by an asynchronous agent using GPT-4o [6], which extracts every possible entity and relationship. This reveals entities not predefined in any ontology that may be crucial. The ontology dynamically adapts based on the classified document type and legal domain. The output is a schema of node and relationship type lists.

2) *Ontology Refinement*: The extracted schema is compared against a predefined reference ontology (consistent across Slovak law, e.g., `Person`, `Law`, `Paragraph`) or the existing graph schema. This ensures consistency, prevents semantically similar but differently named types (e.g., `REGULATES` vs. `REGULATED`), and eliminates duplicates. Gemini-2.5-Flash is used for this reasoning task.

3) *Schema-Guided Extraction (SDE)*: The most precise step: each chunk is processed in parallel with the refined ontology provided to the LLM. The agent extracts consistent entities and relationships, returning `GraphDocument`



Fig. 1. Knowledge graph structure: document, chunk, and entity relationships.

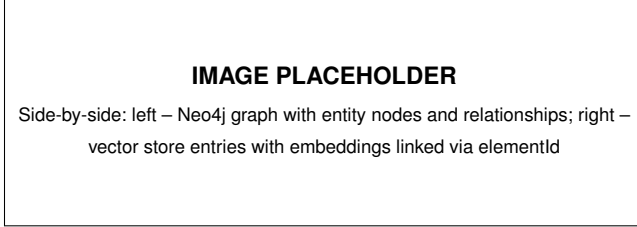


Fig. 2. Dual database architecture: graph DB and vector store linked via elementId.

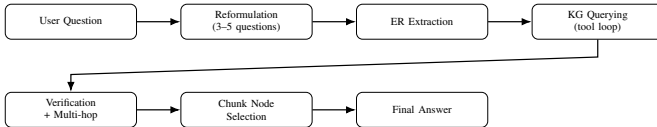
objects. The result is a rich, consistent knowledge graph where chunks link to documents via `IN_DOCUMENT` and to entities via `HAS_ENTITY` relationships.

C. Database Insertion

Data are inserted into two databases. The **graph database** (Neo4j) stores `GraphDocument` objects where each entity has a unique `elementId`, labels, and properties; relationships contain type, source, and target identifiers. The **vector database** (Neo4jVector) stores entity and relationship embeddings generated by OpenAI’s `text-embedding-3-large` model. These embeddings enable semantic search via cosine similarity and are linked to the graph database through shared `elementId` values.

III. SEARCH AND RETRIEVAL

To answer user questions, the system employs a hybrid GraphRAG approach combining knowledge graph traversal with semantic vector search. The pipeline executes the following steps:



Question Reformulation. The LLM generates 3–5 reformulations of the original question, each phrasing the key information differently to maximize retrieval coverage from the graph database.

Entity-Relationship Extraction. For each reformulated question, the LLM extracts all possible entities and relationships using open-domain extraction, regardless of type.

Knowledge Graph Querying. For each extracted entity and relationship, semantically similar counterparts are found in the vector database via `elementId` linkage. The LLM generates

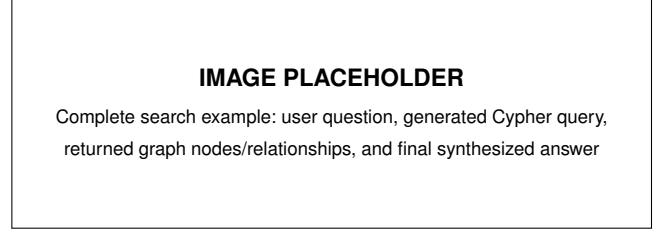


Fig. 3. End-to-end search pipeline example.

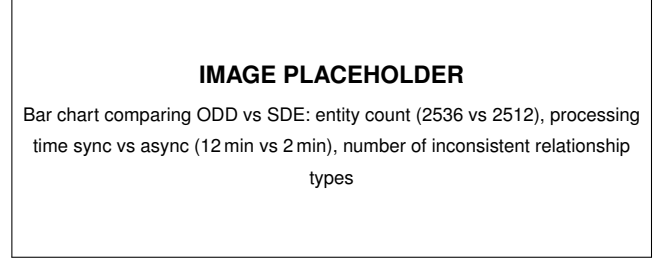


Fig. 4. ODD vs. SDE: entity count and processing time comparison.

Cypher queries iteratively, executing them through a tool with restricted database access.

Verification and Multi-hop Reasoning. The LLM analyzes query results for relevance and completeness. If information is fragmented, multi-hop reasoning activates: traversing indirect entity relations (e.g., person → contract → law), supplementing context via further iterations, and eliminating hallucinations by verifying facts against the graph structure.

Chunk Node Selection. For all discovered nodes, associated `Chunk` nodes are retrieved via `HAS_CHUNK` relationships, providing source text context.

Final Answer Generation. The answer is synthesized from the original question, reformulations, graph results, chunk texts, and the system prompt, processed by Gemini-2.5-Flash with its large context window.

IV. TESTING AND RESULTS

The system was tested on *Romeo and Juliet* as a controlled case for both processing and search evaluation.

A. Processing Evaluation

Initial open-domain extraction yielded a rich but inconsistent graph. Schema-guided extraction using a refined schema from the open-domain results produced a consistent graph with approximately the same entity count:

$$N_{\text{ODD}} = 2536 \approx N_{\text{SDE}} = 2512 \quad (1)$$

Converting from synchronous to asynchronous processing reduced time approximately 6×:

$$T_{\text{sync}} = 12 \text{ min}, \quad T_{\text{async}} = 2 \text{ min} \quad (2)$$

IMAGE PLACEHOLDER

Neo4j subgraph: Juliet and Romeo nodes connected by LOVES, linked to Capulet and Montague house nodes via BELONGS_TO

Fig. 5. Query result: cross-house relationships between Juliet and Romeo.

B. Search Evaluation

An iterative querying agent with restricted graph database access was used to verify data quality. Query processing time ranged from 20 seconds to 4 minutes. The agent consistently found relevant data and produced accurate answers.

For the question “Which characters belonging to the Capulet house are in love with a character from the Montague house?”, the system correctly identified Juliet. However, open-domain extraction revealed relationship inconsistency: instead of a single LOVES relationship, five variants existed (LOVES, IN_LOVE_WITH, EXPRESSES_FEELING_FOR, LOVER, LOVE), demonstrating the importance of ontology refinement.

Testing on this controlled dataset helped refine the system before deployment on Slovak legal documents from Slov-Lex [7].

V. CONCLUSION

This work presented a system for automated knowledge graph construction from legal documents using a hybrid LLM-based extraction pipeline. The combination of open-domain detection with schema-guided extraction produces rich and consistent knowledge graphs. The GraphRAG-based search pipeline leverages both graph traversal and semantic similarity for accurate question answering.

Testing demonstrated that the hybrid approach maintains entity richness while improving consistency, and asynchronous processing reduces computation time by 6×. Future work includes prompt optimization, a web-based user interface, and extending coverage to the full breadth of Slovak legal documentation.

ACKNOWLEDGMENT

This paper was supported by DISIC (09I05-03-V2), NEXT (ERASMUS-EDU-2023-CBHE-STRAND-2), EULiST (ERASMUS), CYB-FUT (ERASMUS+), InteRViR (VEGA 1/0605/23).

REFERENCES

- [1] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, “A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly,” *High-Confidence Computing*, vol. 4, no. 2, 2024, Art. no. 100211.
- [2] D. Edge *et al.*, “From local to global: A graph RAG approach to query-focused summarization,” arXiv preprint arXiv:2404.16130, 2024.
- [3] Neo4j, Inc., “Neo4j graph database,” [Online]. Available: <https://neo4j.com/>
- [4] LangChain, Inc., “LangChain documentation,” [Online]. Available: <https://docs.langchain.com/>
- [5] Q. Xia, H. Ye, and M. Sap, “ODKE+: Ontology-guided open-domain knowledge extraction with LLMs,” arXiv preprint arXiv:2509.04696, 2025.
- [6] OpenAI, “GPT-4o model card,” 2024. [Online]. Available: <https://openai.com/>
- [7] Slov-Lex, “Legal and information portal,” Ministry of Justice of the Slovak Republic. [Online]. Available: <https://www.slov-lex.sk/>